Graph Neural Networks

Harry Wik, Victor Pekkari

March 2026

CONTENTS

1 INTRODUCTION 4
- 1.1 Research questions 4
- 1.2 Scope and limitations 5
- 1.3 Thesis outline 5

2 BACKGROUND 6
- 2.1 Graphs and graph representation learning 6
  - 2.1.1 Transductive nature of traditional GRL 7
- 2.2 Neural networks 7
- 2.3 Graph neural networks 8
  - 2.3.1 Message passing 8
  - 2.3.2 Implications for large-graph training 8
- 2.4 Graph neural networks for large graphs 9
  - 2.4.1 Training Under Memory Constraints 9
  - 2.4.2 Subgraph sampling 10
- 2.5 PyTorch Geometric 10
- 2.6 Neo4j and graph databases 11
  - 2.6.1 Alternatives to Neo4j 11
  - 2.6.2 Querying Neo4j with Cypher and Procedures 11
  - 2.6.3 Data Retrieval from Neo4j 11
  - 2.6.4 Cypher Query Execution Pipeline 12

3 RELATED WORK 13
- 3.1 Subgraph sampling 13
- 3.2 Distributed GNN systems 13
- 3.3 Database-backed GNN pipelines 13

4 METHOD 15
- 4.1 Experimental methodology 15
- 4.2 Baseline graph-DB backed GNN system 15
- 4.3 System configurations 15
- 4.4 Equivalence constraint 16
- 4.5 Performance evaluation 16
- 4.6 Benchmarking procedure 16
- 4.7 Datasets 16
- 4.8 Graph neural network model 17

5 SYSTEM DESIGN 18
- 5.1 Baseline architecture 18

5.2 Improved architecture 18
5.3 Sampling Algorithms 18
5.3.1 Neighbor Sampling 18
5.3.2 GraphSAINT random-walk sampling 19
5.3.3 Graph Representation in Neo4j 19
5.3.4 PyG integration 19
5.3.5 Profiling and logging 20

6 RESULTS 21
6.1 RQ1: Bottlenecks 21
6.1.1 Training phase vs sampling phase 21
6.1.2 Sampling subphases 21
6.1.3 Feature vector datatype 21
6.1.4 Impact of: Multisampler parallelism 21
6.1.5 Impact of: Caching to reduce feature fetch overhead 21
6.2 RQ2: Sampling 21
6.2.1 Neo4j: Neighborsampling vs Graphsaint 21
6.2.2 Neo4j sampling vs PyG sampling 21

7 DISCUSSION 22
7.1 Optimal system implementation 22

8 CONCLUSION 23
8.1 RQ1 - Bottleneck 23

1 INTRODUCTION

Graph structures are a natural way to model entities and their relationships in several important domains. Such models are applied in diverse domains, including predicting chemical bonds in medical science and detecting fraud in finance. The prevalence of graph-structured data across many domains creates a need for efficient and accurate methods to model and analyze graphs.

graph neural networks (GNNs) have emerged as a leading approach for representation learning on graphs, combining the structural intuition behind classical graph algorithms with the expressiveness and inductive capabilities of neural networks.

However, GNNs introduce a challenge absent in traditional neural networks: message passing creates growing dependencies between nodes during training and inference.

Existing approaches for scaling GNNs to large graphs rely on partitioning the graph, sampling subgraphs, and distributing computation across machines. These approaches often suffer from high communication overhead between machines, and from inefficient sampling procedures that can take longer than the actual model training.

In practice, many large graphs are already stored in graph databases before they are loaded into workers for GNN training or inference. The characteristics of fast graph traversal and practically unlimited memory in graph databases make them a promising component in a GNN training and inference pipeline for large graphs. While it has been shown that it is possible to use a graph database as a backend for GNN training, prior work has mainly demonstrated feasibility rather than performance competitiveness with state-of-the-art distributed systems. It is therefore still unclear (i) what the bottlenecks in graph-database-backed training and inference are, (ii) what an efficient graph-database-based pipeline would look like, (iii) whether and when graph-database-backed training pipelines might be a better choice than distributed systems, and (iv) how inference should be done with the graph stored in a graph-DB.

This thesis investigates whether a graph database can serve as an efficient and scalable backend for GNN training and inference on large graphs. To answer this, we implement and evaluate a Neo4j-backed pipeline integrated with PyTorch Geometric, analyze its main bottlenecks, compare sampling strategies, and study how the usefulness of this approach depends on graph size and inference setting.

1.1 RESEARCH QUESTIONS

This thesis addresses the following research questions:

1. **RQ1 — Bottlenecks:** What are the primary bottlenecks in GNN training when using a graph database for graph storage, and to what extent can they be mitigated?

2. **RQ2 — Sampling:** Which subgraph sampling strategies are most efficient when the graph is stored in a graph database?
3. **RQ3 - Inference:** What is the optimal method for doing inference on new nodes when the graph is stored in a graph database?
4. **RQ4 — Graph size:** For which graph sizes does a graph database-based storage approach provide the greatest benefits?

1.2 SCOPE AND LIMITATIONS

The experiments in this thesis are conducted on four datasets of varying sizes using a two-layer Graph Convolutional Network (GCN) for node classification. The main emphasis is on the training and inference pipeline rather than on model architecture. The study is therefore limited to a specific GNN setting, but targets system-level questions that are relevant to large-scale graph learning more broadly.

1.3 THESIS OUTLINE

Chapter 2 provides background on important topics fundamental for understanding the the rest of the thesis, such as GNNs, neural networks, graph representation learning, neo4j and pytorch geometric. Chapter 3 reviews related work on GNNs applied to large graphs. Chapter 4 describes the methodology used during the experiments. Chapter 5 outlines the implemented GNN pipelines. Chapter 6 describes the results, chapter 7 is discussion, and chapter 8 is conclusion.

2 BACKGROUND

Graphs in mathematics are used to model pairwise relationships between objects. Objects are modeled as nodes $\mathcal{V}$, and their relations are called edges $\mathcal{E}$. Many domains benefit from modeling data as objects (nodes) and relationships (edges). Consider, for example, social networks, protein–drug interactions, or railway networks. Studying graph structure and node features allows us to predict properties of the graph. For example, predictions may concern how a graph evolves over time. In fraud detection, nodes can represent accounts and edges transactions. By modeling typical graph evolution, such as which transactions and accounts are added, it becomes possible to detect anomalous transactions that deviate from expected patterns.

When making predictions about graphs it is often good to start by creating an embedding of the graph. The optimal embedding depends on the downstream task. There are several ways to embed a graph, transductively with traditional graph representation learning (GRL) approaches, and inductively with GNNs.

The purpose of this section is to give a fundamental understanding in topics such as traditional GRL algorithms, neural networks, describe how GNNs combine ideas from traditional graph representation learning and neural networks. From traditional GRL GNNs inherit the idea that node embeddings should reflect graph structure, while from neural networks they adopt a learnable, parameterized function. Challenges working with GNNs on big datasets, that do not exist in traditional neural networks too is brought up in this section.

Lastly a basic introduction in `PyTorch Geometric`, the machine learning library used during experiments, and `Neo4j`, a graph database that provides useful graph operations for training GNNs, which is used as the graph storage during the experiments.

2.1 GRAPHS AND GRAPH REPRESENTATION LEARNING

In graph representation learning the aim is to find a representation of the graph that encodes it's structure. Embeddings like this are often used in downstream tasks such as node classification, link prediction, or clustering. Finding a good embedding is somewhat dependent on what the downstream task is.

The simplest way of constructing node embeddings is to manually choose a node statistic to encode the node (e.g. node degree or centrality measures). A more advanced group of methods involve factorizing the adjacency matrix into node-node similarity. Another class of methods is based on random walks. These methods learn embeddings by maximizing co-occurrence between nodes appearing in the same random walks, and can be shown to implicitly perform matrix factorization.

2.1.1 *Transductive nature of traditional GRL*

A common feature throughout all graph representation learning algorithms above, using node statistics, matrix factorization, or random walks, is that no weights are shared between nodes, no node-embedding generating function is learned, only node embeddings for the current nodes in the graph. This means that if new nodes are introduced to the graph, not only isn't there a way to compute their embeddings, the embeddings for the old nodes are not accurate anymore (the later also happens if nodes disappear from the graph).

Consider a simplification of the optimization problem for the random walk algorithms.

$$\min_{\{z_x \,:\, x \in \mathcal{V}\}} \sum_{(u,v)\in\mathcal{D}} -\log\left(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)\right)$$

where nodes $\mathcal{D}$ is a set of pair-of-nodes encountered on random walks, and $\mathcal{V}$ is the set of notes in the graph.

The optimization problem above is an example of how it is the actual node embeddings that are optimized (and therefore learned), rather than some embedding generating function. This is what makes all traditional representation learning algorithms transductive in nature.

Consider a social network where people are nodes, and their relations are edges. if a transductive graph representation is used, it is the feature vectors for specific people in the graph. If more people are introduced to the network, the feature vector for every person has to be recomputed.

2.2 NEURAL NETWORKS

Neural networks are parameterised functions that can learn a function mapping inputs to outputs. Given an input x, produces an output $\hat{y}$ given a function $f_\theta(x)$, where θ are the parameters (also known as weights) of the model.

Optimizing the weights θ of the model include minimizing a loss function $\mathcal{L}$ which measures how off the model prediction is from the desired target values. This optimization is usually done using some gradient based method.

A key property of neural networks is that they are inductive: instead of learning representations tied to specific data points, they learn a general function that can be applied to unseen inputs. This enables generalization beyond the training data.

Standard neural network training further relies on the assumption that datapoints are independent. Under this assumption, the loss can be approximated using mini-batches of data, allowing efficient and scalable training. However, this assumption does not hold for graph-structured data, where datapoints (nodes) are explicitly dependent on each other through the graph topology. This limitation motivates the development of graph neural networks, which extend neural networks to account for relational dependencies.

Standard neural network training further relies on the assumption that datapoints are independent. Under this assumption, the loss can be approximated using mini-batches of data, allowing efficient and scalable training. However, this assumption does not hold for graph-structured data, where datapoints (nodes) are explicitly dependent on each other

through the graph topology. This limitation motivates the development of graph neural networks, which extend neural networks to account for relational dependencies.

2.3 GRAPH NEURAL NETWORKS

Graph neural networks (GNNs) are neural networks designed for graph-structured data. Like traditional graph representation learning methods, they aim to produce node representations that reflect graph structure. Unlike shallow graph embedding methods, however, GNNs learn a shared parameterized function that generates node representations from node features and graph connectivity. This makes GNNs inductive: once trained, the same model can be applied to unseen nodes, and in some cases to entirely new graphs drawn from a similar distribution.

A central idea in GNNs is *message passing*. Instead of computing a node representation only from the node's own features, a GNN updates each node by aggregating information from its neighbors. By stacking several such layers, the model can incorporate information from increasingly distant parts of the graph.

2.3.1 *Message passing*

At message passing layer k, the representation of node v is updated as

$$\mathbf{x}_v^{(k)} = \phi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(k)}\left(\mathbf{x}_v^{(k-1)}, \mathbf{x}_u^{(k-1)}\right)\right),$$

where $\mathcal{N}(v)$ denotes the neighbors of node v, $\psi^{(k)}$ is a message function, $\bigoplus$ is a (most often permutation-invariant) aggregation operator such as sum or mean, and $\phi^{(k)}$ is an update function.

After K layers, the representation of a node depends on information from its K-hop neighborhood. This allows GNNs to exploit both node features and graph structure when solving downstream tasks such as node classification.

2.3.2 *Implications for large-graph training*

The message-passing mechanism also creates the main computational challenge in GNN training. In standard neural networks, datapoints in a mini-batch are usually treated as independent, so the loss and gradient can be computed from the batch alone. In a GNN, this is no longer true: the representation of a node depends on its neighbors, whose representations in turn depend on their neighbors. As a result, computing the output for a batch of seed nodes generally requires access to a larger induced subgraph around those nodes.

This dependency grows with the number of GNN layers. For a K-layer model, each seed node may depend on a K-hop neighborhood, which can become very large in real-world graphs. Training on the full graph is therefore often infeasible for memory and computation reasons, especially for large graphs. To address this, practical GNN pipelines typically rely on subgraph or neighborhood sampling, where only a bounded part of the neighborhood is retrieved for each mini-batch.

2.4 GRAPH NEURAL NETWORKS FOR LARGE GRAPHS

In typical neural networks, datapoints are either independent or have dependencies with a clear and structured form. This structure is determined by the architectural inductive bias, which makes it straightforward to derive which datapoints influence each other. For example, in RNNs, t_n depends on $\{t_{n-1}, t_{n-2}, \dots\}$, while in convolutional neural networks (CNNs), a pixel $p_{x,y}$ depends only on a local neighborhood $\{\, p_{i,j} \mid |i-x| \leqslant r,\ |j-y| \leqslant r \}$ in the same image. In both cases, the set of dependent datapoints is explicitly defined and bounded.

In contrast, graph neural networks lack such explicit structural assumptions. The dependency structure is determined by the graph itself, making it harder to determine which datapoints influence a given node. Moreover, these dependencies are not only less structured, but also significantly larger. The representation of a node depends on its k-hop neighborhood, which can grow exponentially with k. As a result, each datapoint in a GNN may depend on orders of magnitude more datapoints than in models such as CNNs, where dependencies are limited to small local regions.

2.4.1 *Training Under Memory Constraints*

In standard neural networks, the independence (or structured-bounded dependence) between datapoints allows training on datasets larger than available memory. Data can be processed in virtual mini-batches $\{d^{(1)}, \dots, d^{(n)}\}$ such that

$$\sum_i d_x^{(i)} < Y,$$

where Y is the available memory. Crucially, the gradient computed on a batch depends only on the datapoints within that batch. This assumption breaks for GNNs as the output for one node is dependent on an un-bounded number of neighbors. The gradient for the seed nodes in the batch can't be computed accuraretely without their neighbors. To solve this the neighbors of the seed nodes are fetched in the beginning of every batch, to build a sub-graph around the seed nodes so that the gradient for that batch can be computed accuretly.

Algorithmus 1 : Mini-batch GNN Training (Single Epoch)

```
// Mini-batch training with neighbor-sampled subgraphs
Input : f_θ, V
Output : θ
for each batch B ⊆ V do
    G_B ← ∅
    for each seed node v ∈ B do
        (X_v, y_v, E_v) ← SAMPLE(v)
        G_B ← G_B ∪ (X_v, y_v, E_v)
    ŷ ← f_θ(X_B, E_B)
    L ← CROSSENTROPY(ŷ, y_B)
    θ ← θ − η∇_θ L
return θ
```

2.4.2 *Subgraph sampling*

To address this, GNN training typically relies on subgraph sampling. Instead of using full neighborhoods, a bounded subset of neighbors is sampled for each node. Given a batch of seed nodes, a subgraph is constructed by sampling up to a fixed number of neighbors at each hop. This limits the size of the computation graph and enables mini-batch training.

A common approach is layer-wise neighbor sampling, where upper bounds are defined as $[k_0, k_1, \ldots, k_{K-1}]$. At layer i, at most k_i neighbors are sampled per node. If the number of neighbors exceeds this bound, a subset is selected, typically uniformly at random.

While sampling makes training tractable, it introduces an approximation of the true neighborhood aggregation. Consider the exact aggregation for a node v, using the mean of node-neighbor feature-vectors as the aggregation function:

$$h_v = \sigma\left(\frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} x_u\right),$$

where σ is a nonlinear activation function.

With sampling, we instead compute:

$$\tilde{h}_v = \sigma\left(\frac{1}{k} \sum_{u \in \tilde{\mathcal{N}}(v)} x_u\right),$$

where $\tilde{\mathcal{N}}(v)$ is a sampled subset of neighbors.

Although the sampled mean is an unbiased estimator of the true mean,

$$\mathbb{E}\left[\frac{1}{k} \sum_{u \in \tilde{\mathcal{N}}(v)} x_u\right] = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} x_u,$$

the presence of the nonlinearity introduces bias:

$$\mathbb{E}[\tilde{h}_v] = \mathbb{E}[\sigma(\cdot)] \neq \sigma(\mathbb{E}[\cdot]) = h_v.$$

This discrepancy arises because σ is nonlinear, and therefore expectation does not commute with the activation function. In practice, this bias is often small and does not prevent convergence, especially for shallow GNNs.

2.5 PYTORCH GEOMETRIC

PyTorch Geometric is a graph machine learning library built on top of pytorch. It provides implementations of various message passing layers, as well as abstractions for storing the graph.

- `FeatureStore:` which defines how the node features are stored and retrieved.
- `GraphStore:` which defines how the graph is traversed, and how edges are stored.
- `BaseSampler:` which defines the exact how the neighborhood sampling is done, this is tightly coupled with graphstore.
- `NodeLoader:` takes instances of implementations of FeatureStore, GraphStore, and BaseSampler tand provides a dataloader.

2.6 NEO4J AND GRAPH DATABASES

Databases are systems used to efficiently store and fetch data. Instead of scanning raw files, databases provide a way to find relevant data with low latency. Traditional databases are relational, and the data is stored in the form of tables of rows and columns. Relationships between objects are stored through foreign keys, which leads to expensive join operations when retrieving relational data. This model is not well suited for queries that involve repeated traversal of relationships, which is something needed for fast subgraph sampling in GNN training.

Graph databases such as Neo4j are, in contrast, optimized to have efficient operations on graphs. This is done by explicitly modeling nodes and edges, making relationships first class citizens. This allows queries that follow connections, such as multi-hop traversals, to be expressed and executed more naturally and efficiently. In an ETL-pipeline to a GNN-model it is of necessity that the database supports efficient operations, mainly retrieval, and scalability in the form of horizontal scaling via clustering as well as vertical scale-up of the machines running the database instance(s).

2.6.1 *Alternatives to Neo4j*

Although Neo4j is a market leader, capturing the largest market share in the graph database space, there are several alternatives. Amazon Neptune, Azure Cosmos and Google Spanner Graph are examples that mainly compete with Neo4j by tight integration to the respective cloud platforms: AWS, MS Azure and Google Cloud. Other examples include Memgraph and Tigergraph.

2.6.2 *Querying Neo4j with Cypher and Procedures*

Cypher is a declarative query language for graph databases where the queries are expressed as pattern matches over the nodes and edges in the graph. The travel planner translates the cyhper code into execution plans optimized for traversal and lookup. In this work, Cypher is used to express neighborhood expansion queries required for subgraph sampling.

Procedures extend cypher with imperative logic that cannot be expressed in cypher. All Procedures are defined in java within the database engine, allowing custom traversal strategies, early termination, and more fine-grained control over execution. This makes procedures a relevant alternative when trying to optimize highly specialized cypher queries, procedures are something that is explored just as well as cypher when fetching neighbors of the seed nodes.

2.6.3 *Data Retrieval from Neo4j*

Data is retrieved in batches determined by the `fetch_size`. A smaller `fetch_size` results in more round trips between the client and the database. For each batch, a number of rows corresponding to the `fetch_size` are first located in the database and then serialized before being transmitted over the network. The serialization speed depends on the underlying data format; for example, byte arrays are typically faster to serialize than more complex

structures. While one batch is being transmitted, the next batch is prepared in the database, serialized, and subsequently sent to the client.

2.6.4 *Cypher Query Execution Pipeline*

The incoming Cypher string is parsed into an Abstract Syntax Tree, AST. The AST is then validated, e.g. raises an error if a non-existent label/relationship type/property filter predicate is used. The next step is to make the AST have canonical form. The normalized AST is used to generate one or more logical plans: operator trees that define how to produce the query response. Much like the expression:

$$R := c + a \cdot b$$

Above is a simple algebraic operator tree where the $a * b$ term will be reduced first because of precedence and the result thereof summed together with c and bound to R. The logical plan generated by the Cypher-query will act similarly upon the data source but instead with operators such as `NodeByLabelScan` (lookup nodes with a label) and `Filter` (applies a filter such as `n.age` > 30). One logical plan can be resolved according to many different physical plans all yielding the same output given the input. A simpler query is likely to only have one physical plan whereas a more complex query could have a multitude. In the case of multiple physical plans, they are compared against each other in such a way that a cost metric is minimized. Examples of elements weighed into the cost optimization are metadata such as how many nodes have a certain label and whether to utilize an index lookup. After the cost optimization one physical plan is promoted to be the execution plan. The execution compiles optimized bytecode that can be used (and reused). To answer the query in a timely manner, the execution step can have a parallel runtime.

Some features are restricted to paying database clients i.e. clients that run 'Neo4j Enterprise' rather than 'Neo4j Community'. Parallelism is one such example along with clustering, extra constraints, online backups and Role Based Access Control.

Regardless of parallelism, the execution first attempts to retrieve from the page cache, in RAM. The nodes and relations required to resolve the query are either found in cache through pointer chasing or a page fault occurs and data needs to be loaded from non-volatile memory for example an NVMe-raid disk. The exact method of retrieval depends on the storage [1]format. The current default is block format.

After data has been localized, it needs to be sent back to the client. First, the binary representations are cast to proper types. Next is the removal of non-return variables. Before finally returning data to the client the java classes needs to be serialized. Neo4j has a 'bolt'-server that utilizes PackStream to binary encode the data after which the standard TCP/IP stack takes over and answers the query.

1 https://neo4j.com/docs/operations-manual/current/database-internals/store-formats/

3 RELATED WORK

Existing approaches to scaling GNN training can be broadly categorized as sampling-based methods and distributed systems. Sampling methods reduce computational cost by approximating neighborhoods, while distributed systems improve scalability through parallelism at the cost of communication overhead.

3.1 SUBGRAPH SAMPLING

Sampling methods are widely used when training GNNs because of the exploding neighborhoods that comes from the stacked message passing layers. Sampling based methods samples subgraphs from the larger original graph. This reduces both the computational and memory cost.

GraphSAGE introduced layer-wise neighbor sampling, where a fixed number of neighbors are sampled per node during training. **GraphSAINT** instead samples subgraphs using random walks or node-based sampling strategies, aiming to improve training efficiency and reduce variance.

While these methods make large-scale GNN training tractable, they introduce approximation error due to incomplete neighborhood information, and their performance depends on the efficiency of the sampling procedure.

3.2 DISTRIBUTED GNN SYSTEMS

Another line of work focuses on distributed GNN training, where the graph is partitioned across multiple machines. These systems aim to scale training by parallelizing both computation and data access.

[insert aobut papers here]

Despite their scalability, distributed approaches often suffer from communication overhead between machines and require careful partitioning strategies to balance load and minimize data transfer.

3.3 DATABASE-BACKED GNN PIPELINES

A smaller body of work explores graph-DB backed GNN training. Here the memory storage, and fast graph traversal in graph-DBs is leveraged, instead of storing the graph in-memory across several machines.

Lopushanskyy and Shi demonstrate that it is possible to train GNNs directly on a graph database by retrieving only the required subgraphs during training.

However, existing work in this area has primarily focused on demonstrating feasibility, with limited evaluation of performance compared to state-of-the-art distributed systems. In particular, it remains unclear what the main bottlenecks are and under what conditions database-backed approaches are competitive.

4 METHOD

This research study was conducted at Neo4j, within the Graph Data Science team. It follows an experimental systems-oriented methodology, where different GNN training and inference pipelines are tested in an ablational manor. The primary objective in the experiments is to find a GNN pipeline that keeps functional equivalence to PyG while maximizing throughput.

4.1 EXPERIMENTAL METHODOLOGY

The study is based around a graph-DB backed GNN pipeline that has been proven to have functional equivalence to PyG. Components of the GNN pipeline are modified and benchmarked to identify performance bottlenecks and potential optimizations.

Each experiment isolates a single design choice (e.g., sampling strategy, data retrieval method, or caching mechanism), allowing its impact on performance to be evaluated independently. This enables a systematic exploration of the design space of graph-database-backed GNN training.

4.2 BASELINE GRAPH-DB BACKED GNN SYSTEM

As a reference, a PyG-based training pipeline is used. This pipeline defines the expected behavior of sampling, batching, and model execution, and serves as a ground truth for correctness. The Neo4j-backed implementation replaces the data loading and sampling components of this pipeline while keeping the model architecture and training loop identical.

4.3 SYSTEM CONFIGURATIONS

Experiments are conducted across multiple system configurations, where both hardware setup and pipeline design may vary. These configurations include:

- Different sampling implementations (Cypher queries vs. procedures)
- Variations in data retrieval strategies (batch size, serialization format)
- Use of caching mechanisms in feature retrieval Parallel vs. single-sampler execution

Each configuration represents a distinct point in the design space of graph-database-backed GNN training.

4.4 EQUIVALENCE CONSTRAINT

A key requirement throughout all experiments is that the Neo4j-backed pipeline remains functionally equivalent to the PyG reference implementation. Equivalence is evaluated using:

- Consistency in sampled subgraph structure statistics (e.g. number of sampled edges and nodes at each K-hop)
- Similar training convergence behavior Comparable validation and test accuracy
- Distributional similarity of sampled neighborhoods at each K-hop.

Only configurations that satisfy these criteria are considered valid, ensuring that performance improvements do not come at the cost of altered model behavior.

4.5 PERFORMANCE EVALUATION

The primary performance metric is training throughput, measured as the number of processed seed nodes per unit time. This is complemented by a detailed breakdown of pipeline stages, including:

- Sampling time
- Feature retrieval time

In addition, database-level metrics such as query latency, number of database hits, and serialization overhead are collected to identify system bottlenecks.

4.6 BENCHMARKING PROCEDURE

A dedicated measurement framework is used to collect timing and profiling data during training. Each experiment is run for multiple epochs to ensure stable measurements. Comparisons between configurations are made under identical conditions, including:

- Fixed model architecture
- Consistent hyperparameters
- Identical datasets

This ensures that observed performance differences are attributable solely to system design choices.

4.7 DATASETS

The datasets (consisting of homogenous graphs) to be used for training and evaluating GNNs for node classification were:

- **ogbn-arxiv**, 170k nodes, "The ogbn-arxiv dataset is a directed graph, representing the citation network between all Computer Science (CS) arXiv papers indexed by MAG"

- **ogbn-products**, 2.4M nodes, The ogbn-products dataset is an undirected and unweighted graph, representing an Amazon product co-purchasing network
- **ogbn-papers100M**, 111M nodes, "The ogbn-papers100M dataset is a directed citation graph of 111 million papers indexed by MAG"

4.8 GRAPH NEURAL NETWORK MODEL

All experiments use a two-layer Graph Convolutional Network (GCN). The model applies two `GCNConv` message-passing layers with ReLU activations and a final linear classifier. The architecture is intentionally kept simple to focus evaluation on the data pipeline rather than the model itself. The parameters in the model that varied was the dimension of the first GCNConv layer (same dimension as the dimension of the node feature-vectors, the output dimension (same as the number of classes for the dataset.

5 SYSTEM DESIGN

The system consists of three main components: a Neo4j graph database acting as the graph and feature store, a Python training process that implements the PyG abstract interfaces, and a `NodeLoader` that orchestrates mini-batch construction during training. The implementations `FeatureStore` and `GraphStore` acts as the interface between the training process and the `Neo4j` database. The implementation of `BaseSampler` defines how the subgraph sampling is being done.

5.1 BASELINE ARCHITECTURE

5.2 IMPROVED ARCHITECTURE

5.3 SAMPLING ALGORITHMS

Two sampling algorithms are evaluated:

5.3.1 *Neighbor Sampling*

For each seed node in the mini-batch, the sampler issues a Cypher query that traverses the graph up to K hops, sampling at most $[k_0, k_1, \ldots, k_{K-1}]$ neighbors at each hop. Neighbors exceeding the limit are selected uniformly at random without replacement. This corresponds to the GraphSAGE sampling strategy.

Algorithmus 2 : Neighbor Sampling

Input : Seed nodes S, neighbor limits $[k_0, k_1, \ldots, k_{K-1}]$
Output : Sampled node set $\mathcal{V}_{sub}$

```
V_sub ← S
begin ← 0
end ← |S|
for ℓ ← 0 to K − 1 do
    k ← k_ℓ
    for i ← begin to end − 1 do
        v ← V_sub[i]
        Sample up to k neighbors of v uniformly at random, with or w
        Add sampled neighbors to V_sub if not already present
    begin ← end
    end ← |V_sub|
return V_sub
```

5.3.2 *GraphSAINT random-walk sampling*

Instead of expanding neighborhoods around seed nodes, GraphSAINT constructs a subgraph by first sampling a set of nodes via random walks and then inducing the subgraph over those nodes.

5.3.3 *Graph Representation in Neo4j*

Nodes in Neo4j are stored with a label corresponding to the node type (e.g., `Paper` for citation networks). Each node carries the following properties:

- A unique integer id.
- Pre-computed node feature vectors. The vectors can either be stored as Float64[] or bytes[]
- A target class label for supervised node classification, which is an integer signifying what class the node belongs to
- A split indicator (`train`, `val`, `test`) used to partition the dataset. This can either be a string, or integer.

Edges are stored as typed relationships connecting pairs of nodes. No edge features are used in the experiments.

5.3.4 *PyG integration*

Three PyG abstract classes described in **??** were implemented:

`Neo4jAbstractFS` is an abstract class extending FeatureStore. The purpose of this class is to provide storage of the feature vectors (e.g. a graph database), and to implement an efficient interface for fetching feature vectors. The reason it is kept abstract is that while there are some methods any Neo4j interacting PyG FeatureStore needs (e.g. executing queries) there are also some features that may vary depending on the use case. One example of this is caching. Avoiding the neo4j fetching by storing commonly used features in RAM is a good optimization, but how this caching is done in the feature store is something that should be up to the user. There are also two extensions of the `Neo4jAbstractFS`, which in the `Neo4jCachedFS` and `Neo4jNoCacheFS`.

`Neo4jAbstractGS` an abstract class extending GraphStore, with the main goal of implementing efficient traversal over the graph to enable fast subgraph sampling. It has two implementations `Neo4jMultiGS` and `Neo4jSingleGS` where the difference is how many sampler they are optimized of being able to handle.

`Neo4jSampler` implements the sampling algorithm, and is tightly coupled with the graph store where it executes the sampling algorithm.

5.3.5 *Profiling and logging*

Training logging

Measurements are continously taken during training and stored in a

Query Execution Profiling

Cypher queries are instrumented with the `PROFILE` keyword to collect operator-level statistics: rows produced, DB hits, estimated memory usage, and query execution time. These are aggregated across batches and stored as averaged CSV.

Python-Level Profiling

Call-stack profiles are collected using Python's `cProfile` module and stored as `train_profile.txt`. These are used to identify hot-spots in the Python data pipeline that are not visible in the query-level metrics, such as tensor allocation overhead or unnecessary data copies.

6 RESULTS

6.1 RQ1: BOTTLENECKS

6.1.1 *Training phase vs sampling phase*

6.1.2 *Sampling subphases*

6.1.3 *Feature vector datatype*

6.1.4 *Impact of: Multisampler parallelism*

6.1.5 *Impact of: Caching to reduce feature fetch overhead*

6.2 RQ2: SAMPLING

6.2.1 *Neo4j: Neighborsampling vs Graphsaint*

6.2.2 *Neo4j sampling vs PyG sampling*

7 DISCUSSION

7.1 OPTIMAL SYSTEM IMPLEMENTATION

8 CONCLUSION

8.1 RQ1 - BOTTLENECK

The experiments show that the primary bottleneck is the deser